

Module 7

Orchestration agentique (partie 2)

State management avancé · HIL · CrewAI

Cursus AI Engineer · Neosoft · 1 jour

Positionnement du module

Vous avez (M6) :

- Un agent ReAct LangGraph fonctionnel
- Un graphe avec State, nodes, conditional edges

Ce module ajoute la dimension **production-ready** :

- **Checkpointing** : persister l'état, reprendre après crash
- **Time-travel debugging** : rejouer depuis n'importe quel point passé
- **Human-in-the-loop** : suspendre avant action sensible
- **Garde-fous** : budgets, retries, détection de boucle, dégradation
- Comparaison **LangGraph vs CrewAI** : critères de choix

Objectifs pédagogiques

À l'issue du module, vous serez capable de :

1. **Configurer** un checkpointer (SQLite / PostgreSQL)
2. **Reprendre** une conversation interrompue avec son `thread_id`
3. **Rejouer** un agent depuis un snapshot passé (time-travel)
4. **Implémenter** un point d'approbation humaine via `interrupt_before`
5. **Mettre en place** budgets, timeouts, fallbacks
6. **Choisir** entre LangGraph et CrewAI pour un cas donné

Plan de la journée

Bloc	Contenu	Durée
Matin	Checkpointing & persistance	1h
	Time-travel debugging	30 min
	Human-in-the-loop	1h
	Garde-fous : budgets, retries, fallbacks	30 min
Après-midi	CrewAI : modèle multi-agents	1h
	LangGraph vs CrewAI : analyse comparative	30 min
	Observabilité agents (intro M9)	30 min
	TPs : checkpointing + HIL + CrewAI	1h30

Partie 1

Checkpointing

Pourquoi le checkpointing ?

Un agent en production peut être interrompu à tout moment :

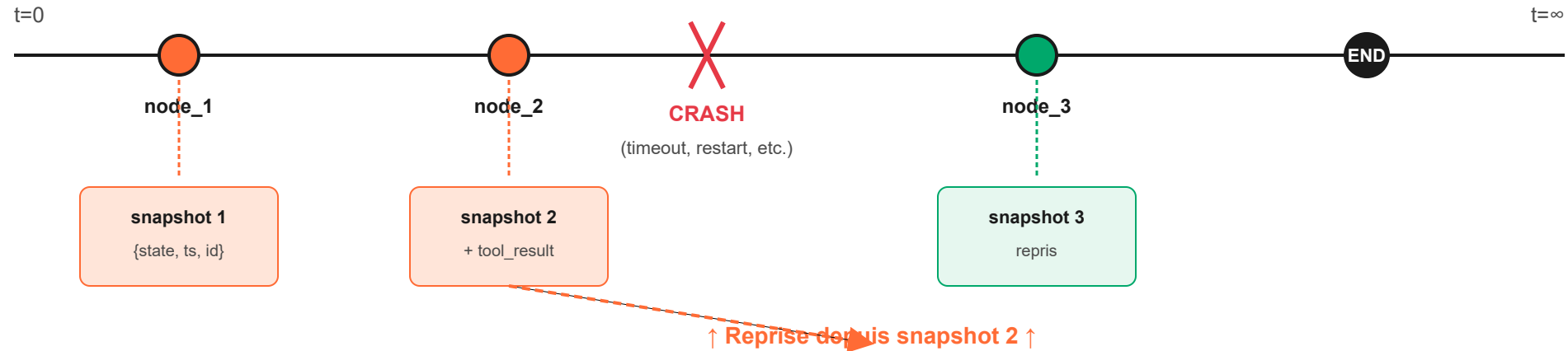
- Le pod K8s redémarre (rolling update)
- L'utilisateur ferme l'onglet, reprend 5 min plus tard
- Un tool met 30 secondes, le client HTTP timeout
- Un appel LLM rate-limite

Sans checkpointing : on repart de zéro à chaque interruption. Conversation perdue, recalculs coûteux.

Avec checkpointing : l'état est sauvegardé après chaque node. On reprend là où on s'est arrêté.

Le mécanisme expliqué

Checkpointing — persistance et reprise d'agents



Backends de checkpointing supportés

SQLite

Dev / single user

PostgreSQL

Production multi-user

Redis

Faible latence, TTL

In-memory (MemorySaver)

Tests / éphémère

SQLite checkpoint — démarrage

```
from langgraph.checkpoint.sqlite import SqliteSaver

checkpointer = SqliteSaver.from_conn_string("checkpoints.db")
agent = graph.compile(checkpointer=checkpointer)

# À chaque invocation, fournir un thread_id
config = {"configurable": {"thread_id": "user_42_session_8"}}

# 1ère invocation : démarre une nouvelle conversation
agent.invoke({"messages": [("user", "Bonjour")]}, config=config)

# 2e invocation avec le MÊME thread_id : reprend l'état
agent.invoke({"messages": [("user", "Et qu'est-ce que tu m'as dit ?")]}, config=config)
```

Le `thread_id` est l'équivalent du `conversation_id`. C'est l'utilisateur de votre app qui le génère.

PostgreSQL checkpoint — production

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://user:pass@host:5432/agents?sslmode=require"

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # crée les tables nécessaires (1ère fois)
    agent = graph.compile(checkpointer=checkpointer)

# Production : pool de connexions, multi-thread
config = {"configurable": {"thread_id": "session_uuid"}}
result = agent.invoke({"messages": [...]}, config=config)
```

Avantages PG :

- Multi-process / multi-pod
- ACID, backups standards
- Indexes sur `thread_id` → lookup $O(1)$

Pour Redis : `from langgraph.checkpoint.redis import RedisSaver` . Idéal pour TTL court.

Lire l'historique d'un thread

```
# État courant
state = agent.get_state(config)
print(state.values)          # State actuel
print(state.next)            # prochain(s) node(s) à exécuter
print(state.config)          # config + checkpoint_id

# Historique complet
for snapshot in agent.get_state_history(config):
    print(f"Step {snapshot.metadata['step']}: {snapshot.next}")
    print(f"  State: {list(snapshot.values.keys())}")
    print(f"  Created: {snapshot.created_at}")
```

*Cet historique est ce qui rend le **time-travel** possible (cf. partie 2).*

Pattern : conversation persistante

```
def chat(user_id: str, message: str) -> str:
    """Une fonction qui maintient le contexte conversationnel par utilisateur."""
    config = {"configurable": {"thread_id": f"user_{user_id}"}}
    result = agent.invoke(
        {"messages": [("user", message)]},
        config=config,
    )
    return result["messages"][-1].content

# Plusieurs jours plus tard, même user_id → contexte préservé
chat("alice", "Bonjour, je m'appelle Alice")          # T0
chat("alice", "Quel est mon prénom ?")                # T+24h → "Alice"
```

Pas besoin de gérer manuellement la mémoire conversationnelle : le checkpointer le fait.

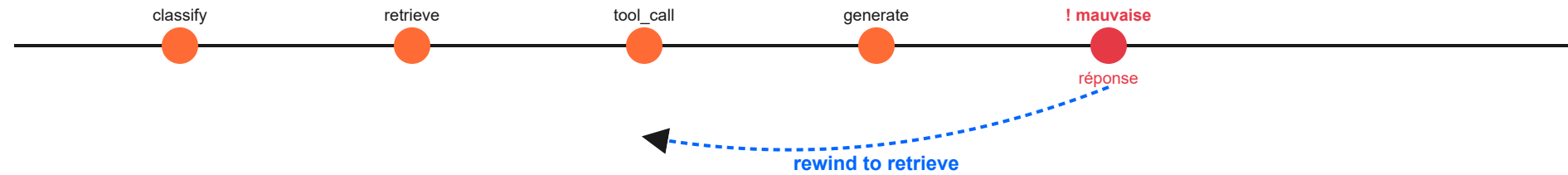
Partie 2

Time-travel debugging

Le concept

Time-travel debugging — rejouer un agent depuis un point passé

Exécution initiale



Replay avec prompt corrigé



Cas d'usage du time-travel

- 1. Debug post-mortem** Reproduire un bug en prod sans relancer toute la chaîne
- 2. A/B testing de prompts** Comparer 2 versions de prompt sur même contexte initial
- 3. Correction utilisateur** L'utilisateur reformule, on repart d'un état précis (pas du début)

Reprendre depuis un snapshot précis

```
# Lister l'historique
history = list(agent.get_state_history(config))

# Reprendre depuis la 3e étape (tool_call)
target_snapshot = history[2] # ordre antichronologique : 0 = le plus récent
target_config = target_snapshot.config

# Replay : continuer comme si on était à ce point
agent.invoke(None, config=target_config)
```

`invoke(None, config=...)` reprend depuis l'état pointé par le `checkpoint_id` dans `config`.

Modifier l'état avant replay

```
# Récupérer le snapshot
target_snapshot = history[2]

# Patcher l'état (ex : corriger une mauvaise classification)
agent.update_state(
    target_snapshot.config,
    {"intent": "factual"}, # corrige le champ
)

# Récupérer la nouvelle config (avec un nouveau checkpoint_id)
new_config = agent.get_state(config).config

# Continuer depuis ce nouvel état
agent.invoke(None, config=new_config)
```

Très puissant pour reproduire un bug, tester un correctif sans relancer toute la chaîne.

Cas d'usage concrets

Scénario	Comment time-travel aide
Bug en prod sur conversation X	Charger le thread, rejouer, identifier l'étape en cause
Test de variante de prompt	Repartir d'un snapshot avec un nouveau prompt, comparer
Utilisateur modifie sa question	Reprendre avant <code>classify</code> , pas tout recommencer
QA team analyse un échec	Pas besoin de re-entraîner le LLM, on rejoue à l'identique

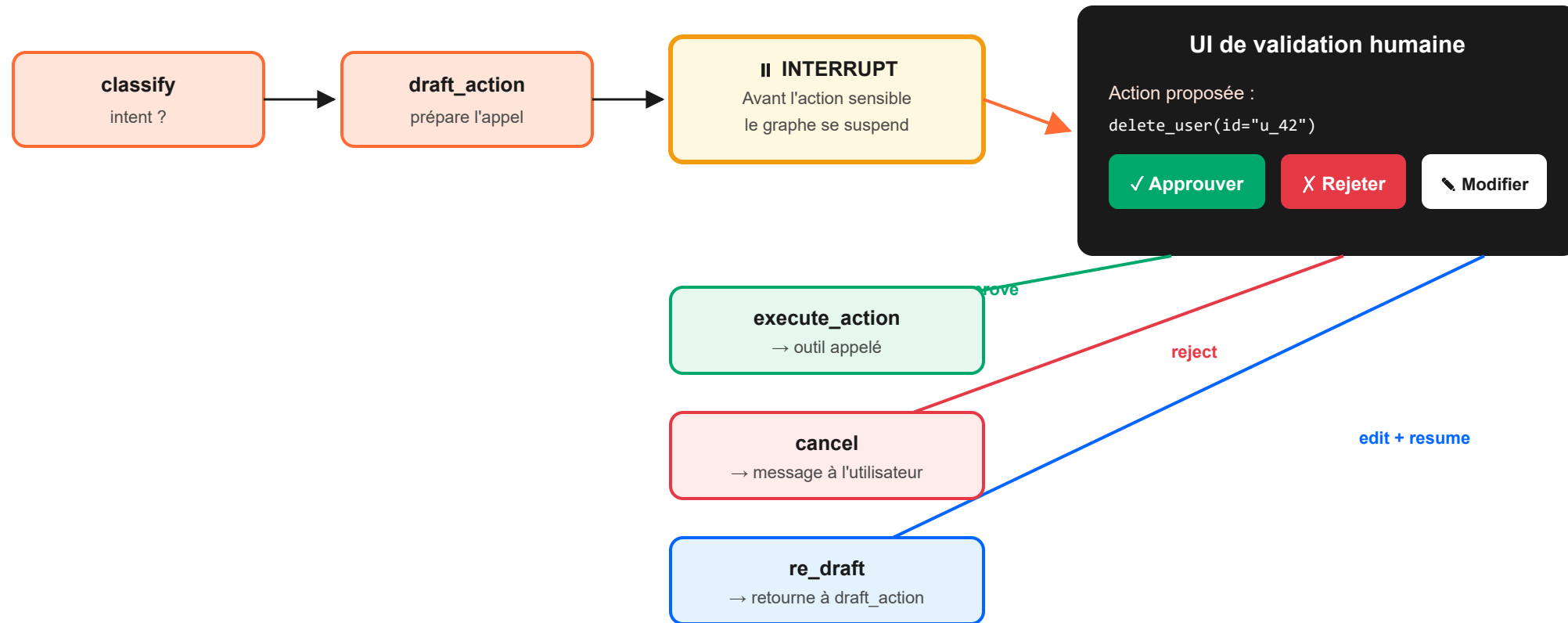
Sans time-travel, chaque debug coûte du temps de re-exécution + des tokens.

Partie 3

Human-in-the-Loop

Le pattern HIL

Human-in-the-Loop — point d'approbation avant action sensible



Implémentation LangGraph : `graph.compile(checkpointer=..., interrupt_before=["execute_action"])`

interrupt_before — implementation

```
from langgraph.checkpoint.sqlite import SqliteSaver

checkpointer = SqliteSaver.from_conn_string("hil.db")
agent = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["execute_action"], # liste de nodes
)

config = {"configurable": {"thread_id": "session_42"}}

# 1. L'agent tourne jusqu'à execute_action puis se suspend
result = agent.invoke({"question": "Supprime user_42"}, config=config)

# 2. État courant : l'agent attend devant execute_action
state = agent.get_state(config)
proposed_action = state.values["draft_action"]
print(f"Action proposée : {proposed_action}") # → l'humain valide

# 3. Si approuvé : reprendre
if user_approves():
    agent.invoke(None, config=config)
```

Modifier l'action avant exécution

L'humain peut **éditer** la proposition avant de la valider :

```
state = agent.get_state(config)
proposed = state.values["draft_action"]

# L'humain modifie la proposition (UI)
modified = {
    "tool": "soft_delete_user", # au lieu de delete_user
    "args": {"user_id": "u_42", "reason": "demande client RGPD"},
}

# Patcher l'état
agent.update_state(config, {"draft_action": modified})

# Reprendre – exécutera modified, pas proposed
agent.invoke(None, config=config)
```

*Pattern courant pour les workflows de **validation 4-eyes** : agent propose, humain valide ou modifie.*

Quels nodes "interrupter" ?

À interrompre :

-  Suppression / modification de données métier
-  Envoi d'email, SMS, notifications externes
-  Création de ticket de support visible client
-  Action irréversible (paiement, archivage)
-  Réponse à une question légalement engageante

À NE PAS interrompre (overhead inutile) :

-  Lecture de données (read-only)
-  Recherche RAG
-  Classification d'intent
-  Génération de brouillon non envoyé

Règle : ***interrupt avant tout side-effect.***

Partie 4

Garde-fous

Les 4 piliers

Gestion d'erreurs et garde-fous d'agents

Budgets d'exécution

max_iterations

Limite le nb de tours ReAct (typiquement 6-12)

max_tokens / session

Plafond de tokens cumulés par session

timeout

Timeout dur par tool et par session

Retries et fallbacks

Retry transient errors

429, 503, network → backoff exponentiel

Fallback model

Si Sonnet down → bascule sur Haiku ou Mistral

Pas de retry sur erreurs métier

400, 422 → propager (l'agent doit corriger)

Détection de boucle

Mêmes args appelés N fois

L'agent répète find_user(id="u_42") sans avancer

Pas de progrès dans le State

Hash du state stable sur 3 tours → break

Action : escalade humaine

Échec contrôlé > boucle silencieuse

Dégradation gracieuse

Réponse partielle

"Voici ce que j'ai trouvé. Je n'ai pas pu vérifier X."

Tool indisponible → continuer sans

Si CRM down, l'agent l'admet et propose un workaround

Toujours fournir un answer

Jamais d'erreur 500 brute exposée à l'utilisateur

Combinés, ces 4 piliers transforment un agent prototype en agent production-ready

Limites d'exécution avec LangGraph

```
# 1. Limite de récursion (= max_iterations)
result = agent.invoke(
    {"messages": [...]},
    config={"recursion_limit": 12}, # plus de 12 nodes traversés → exception
)

# 2. Limite de tokens (custom, à coder dans un node)
def check_budget(state):
    total = sum(estimate_tokens(m) for m in state["messages"])
    if total > MAX_TOKENS_SESSION:
        return {"messages": [AIMessage("Budget dépassé, escalade humaine.")], "stopped": True}
    return {}

# 3. Timeout par tool
import asyncio
async def tool_with_timeout(args):
    return await asyncio.wait_for(slow_tool(args), timeout=10)
```


Détection de boucle

```
def detect_loop(state) -> bool:
    """Détection si les 3 derniers tool_calls sont identiques."""
    messages = state.get("messages", [])
    tool_calls = [
        m for m in messages
        if hasattr(m, "tool_calls") and m.tool_calls
    ]
    if len(tool_calls) < 3:
        return False

    last_3 = tool_calls[-3:]
    fingerprint = lambda m: tuple((c["name"], str(c["args"]))) for c in m.tool_calls
    return fingerprint(last_3[0]) == fingerprint(last_3[1]) == fingerprint(last_3[2])

def safety_check(state):
    if detect_loop(state):
        return {"escalate": True, "reason": "loop detected"}
    return {}
```

Fallback de modèle

```
from langchain_anthropic import ChatAnthropic
from langchain_openai import ChatOpenAI

primary = ChatAnthropic(model="claude-sonnet-4-6", timeout=20)
fallback = ChatOpenAI(model="gpt-4o-mini", timeout=20)

def call_llm_with_fallback(messages):
    try:
        return primary.invoke(messages)
    except (TimeoutError, APIError) as e:
        logger.warning(f"Primary failed: {e}, falling back")
        return fallback.invoke(messages)

# LangChain fournit aussi un helper natif
robust_llm = primary.with_fallbacks([fallback])
```

*Important : le fallback doit être **suffisamment proche** en capacités. Pas de bascule Sonnet → Haiku silencieuse.*

Partie 5

CrewAI

Le modèle CrewAI en 1 minute

CrewAI propose une abstraction "équipe d'agents" :

```
from crewai import Agent, Task, Crew

researcher = Agent(
    role="Senior Research Analyst",
    goal="Trouver les dernières infos sur {topic}",
    backstory="Vous êtes un analyste expérimenté...",
    tools=[search_tool],
)

writer = Agent(role="Tech Writer", goal="Rédiger un article de qualité...", ...)

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process="sequential", # ou "hierarchical"
)
```

*Productif pour démarrer, abstraction claire pour les équipes. **Mais** : moins de contrôle granulaire que LangGraph.*

CrewAI — process séquentiel vs hiérarchique

Sequential : tasks exécutées dans l'ordre, $\text{output}_n \rightarrow \text{input}_{\{n+1\}}$

```
crew = Crew(agents=[...], tasks=[...], process=Process.sequential)
```

Hierarchical : un Manager Agent (auto-créé) délègue aux autres

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, writer, reviewer],
    tasks=[task],
    process=Process.hierarchical,
    manager_llm=ChatAnthropic(model="claude-sonnet-4-6"),
)
```

Le `manager_llm` doit être un modèle "fort" pour bien orchestrer.

CrewAI — features intégrées

Feature	Description
Memory	Mémoire long terme partagée entre agents (entity-based)
Knowledge	Sources documentaires intégrées par agent
Delegation	Un agent peut "demander de l'aide" à un autre
Async	Tasks parallèles via <code>async_execution=True</code>
Output Pydantic	Tasks typées avec schéma de retour

```
from pydantic import BaseModel

class ArticleOutput(BaseModel):
    title: str
    body: str
    word_count: int

task = Task(description="...", expected_output="...", output_pydantic=ArticleOutput)
```

Partie 6

LangGraph vs CrewAI

Comparatif synthétique

LangGraph vs CrewAI — quand utiliser quoi ?

LangGraph

Graphe d'états explicite

Modèle mental

Workflow comme machine à états :
nodes, edges, transitions explicites

Forces

- ✓ Contrôle total des transitions
- ✓ Checkpointing natif (SQLite, PG, Redis)
- ✓ Time-travel debugging
- ✓ Human-in-the-loop fluide (interrupt)
- ✓ Streaming des étapes intermédiaires
- ✓ Déterministe et testable

Limites

- ✗ Plus verbeux pour cas multi-agents
- ✗ Courbe d'apprentissage State + reducers

Cas idéaux

- Production avec SLA stricts
- Workflow avec branchements complexes
- Persistance, reprise, audit obligatoires

CrewAI

Multi-agents par rôles

Modèle mental

Équipe d'agents spécialisés :
Researcher, Writer, Reviewer, Manager

Forces

- ✓ Abstraction "agent + tâche" naturelle
- ✓ Démarrage rapide (peu de code)
- ✓ Délégation hiérarchique built-in
- ✓ Process séquentiel ou hiérarchique
- ✓ Memory et knowledge sources intégrés

Limites

- ✗ Moins de contrôle granulaire
- ✗ Checkpointing limité
- ✗ Debugging plus opaque

Cas idéaux

- POC, prototypes rapides
- Tâches collaboratives multi-rôles

Critères de choix concrets

Critère	LangGraph	CrewAI
Time-to-first-prototype	● Moyen	● Rapide
Contrôle des transitions	● Total	● Limité
Checkpointing	● Natif	● Limité
Human-in-the-loop	● Natif	● À coder
Multi-agents	● Possible	● Natif
Production stable	● Bon	● En progrès
Communauté & écosystème	● LangChain	● Croissante
Coût ops (debug, monitoring)	● Maîtrisé	● Plus opaque

Recommandation 2026 : LangGraph pour la prod, CrewAI pour des POC où la collaboration multi-rôles est centrale.

Pattern hybride

Rien n'empêche de **combiner** :

```
# LangGraph orchestre le flot global avec HIL et checkpointing
def crew_research_node(state):
    """Un node LangGraph qui appelle une crew CrewAI."""
    crew = Crew(agents=[...], tasks=[...])
    result = crew.kickoff(inputs={"topic": state["topic"]})
    return {"research_output": result.raw}

graph.add_node("research", crew_research_node)
graph.add_edge("research", "human_review") # HIL après recherche
```

Le LangGraph "extérieur" gère l'industrialisation, la CrewAI "intérieure" gère la collaboration.

Partie 7

Observabilité (intro)

Tracer ses agents

Sans tracing, un agent en prod est une boîte noire.

```
# Langfuse – auto-instrumentation
from langfuse.callback import CallbackHandler

handler = CallbackHandler(public_key="pk_...", secret_key="sk_...", host="https://cloud.langfuse.com")

result = agent.invoke(
    {"messages": [...]},
    config={"callbacks": [handler], "configurable": {"thread_id": "..."}},
```

Chaque trace inclut :

- Tous les LLM calls (input, output, tokens, coût)
- Tous les tool calls (args, résultats, durée)
- Le graphe d'exécution (parent/child)
- Les erreurs et retries

M9 (LLMOps) couvre Langfuse + MLflow en profondeur.

Métriques agents en prod

À surveiller en continu :

Métrique	Cible typique	Alerter si
Latence p95	< 5 s	> 10 s
Itérations moyennes	< 4	> 6
Coût / requête	budget défini	> 1.5x baseline
Taux d'escalade humaine	5-15%	> 30% (problème de capacité)
Taux d'erreur tool	< 1%	> 5%
Taux de boucle détectée	< 0.1%	> 1%

Sans dashboard, vous découvrez les problèmes via tickets clients. Trop tard.

Partie 8

TPs

TP 1 — Checkpointing + reprise (30 min)

Objectif : agent persistant qui se souvient des conversations précédentes.

→ Voir `exercices/exo1_checkpointing.md`

TP 2 — Human-in-the-loop (30 min)

Objectif : agent qui propose une action sensible et attend validation humaine.

→ Voir `exercises/exo2_hil.md`

TP 3 — Comparaison LangGraph vs CrewAI (30 min)

Objectif : implémenter la même tâche dans les 2 frameworks et comparer.

→ Voir `exercises/exo3_crewai.md`

Récapitulatif

- ✓ Configurer un checkpointer SQLite/PostgreSQL pour persister les conversations
- ✓ Reprendre une session via `thread_id`
- ✓ Faire du time-travel : modifier un snapshot et rejouer
- ✓ Suspendre un agent avant une action sensible (HIL)
- ✓ Mettre en place budgets, timeouts, fallbacks, détection de boucle
- ✓ Choisir entre LangGraph et CrewAI selon le cas d'usage
- ✓ Connaître les métriques à monitorer en production

Prochain module (M8) : MCP, Tool Calling et intégration SI.

Questions ?

Merci pour votre attention

Support, démos et exercices : `supports/M07_agents_partie2/`